

_ Rapport de Mini projet _

Application de Gestion des Chambres

Introduction Générale

Dans le cadre de ma formation, j'ai développé une application de gestion des chambres d'un hôtel en utilisant le langage Java avec NetBeans. Le projet respecte les étapes de la méthode **RUP (Rational Unified Process)**, un processus de développement logiciel itératif et centré sur les cas d'utilisation.

Ce mini projet permet à **un administrateur unique** de :

- S'authentifier avec un **nom d'utilisateur et mot de passe**.
- **Gérer les chambres** (ajout, modification, suppression).
- **Voir les chambres** disponibles avec des détails et des filtres.

CHAPITRE 2 :Inception – Définition du projet

1.1 Objectif

Mettre en place une application desktop permettant à l'administrateur d'un hôtel de gérer les chambres (CRUD) et de consulter leur état, avec un système d'authentification sécurisé.

1.2 Exigences Fonctionnelles

- **R1** : Authentification de l'administrateur.
- **R2** : Gestion des chambres (ajouter, modifier, supprimer).
- **R3** : Affichage des chambres avec filtres (type, prix, disponibilité).

1.3 Exigences Non Fonctionnelles

- Interface intuitive et responsive.
- Réponse instantanée pour l'affichage (< 1s).
- Authentification sécurisée (hash + salt).
- Code testé avec JUnit et qualité vérifiée via SonarLint.

CHAPITRE 2 :Élaboration – Conception UML

Administrateur :

- S'authentifie avec un identifiant et un mot de passe.
- Gère les chambres (ajout, modification, suppression).
- Consulte les chambres disponibles avec filtres.

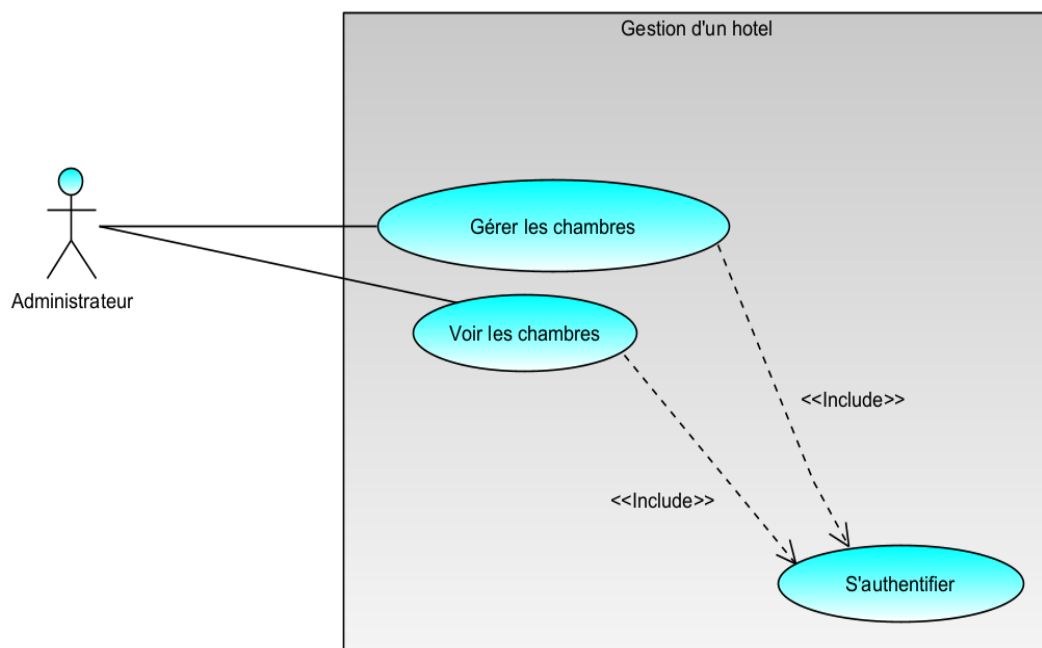
2.1 Cas d'Utilisation Principal :

Cas d'Utilisation	Acteur	Description
UC1 : S'authentifier	Administrateur	Permet à l'administrateur de se connecter au système via identifiants.
UC2 : Gérer les chambres	Administrateur	Ajouter, modifier et supprimer une chambre.
UC3 : Voir les chambres	Administrateur	Consulter la liste des chambres disponibles avec filtres.

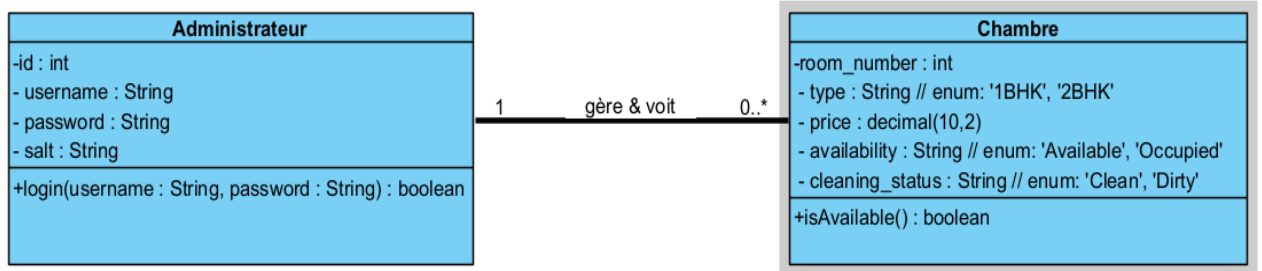
3. Classement des Cas d'Utilisation

Cas d'Utilisation	Priorité	Risque	Itération
UC1 : S'authentifier	Haute	Faible	1
UC2 : Gérer les chambres	Haute	Moyen	2
UC3 : Voir les chambres	Moyenne	Faible	2

2.3 Diagramme de Cas d'Utilisation

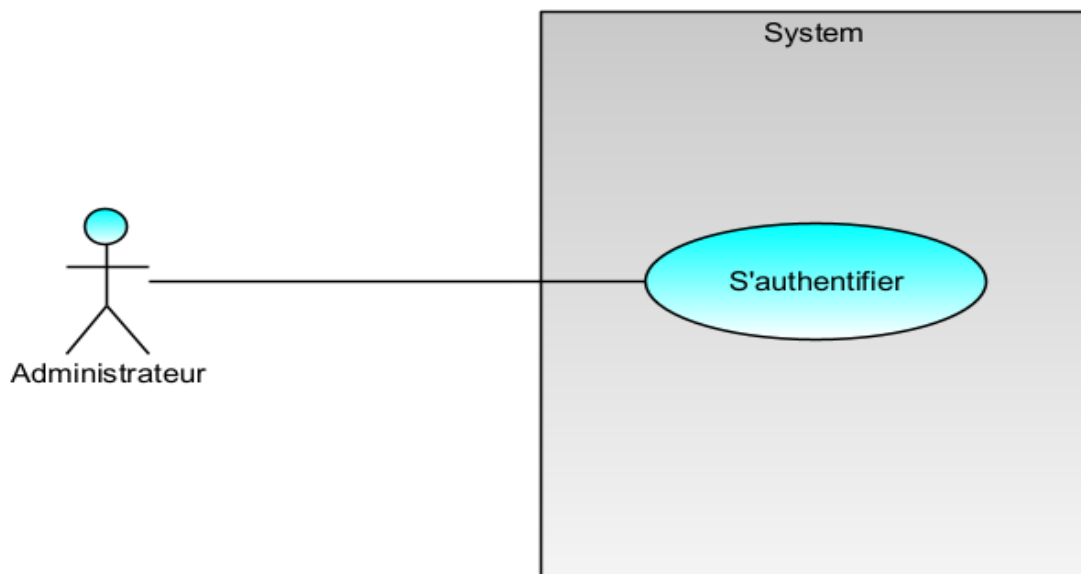


2.4 Diagramme de Classes

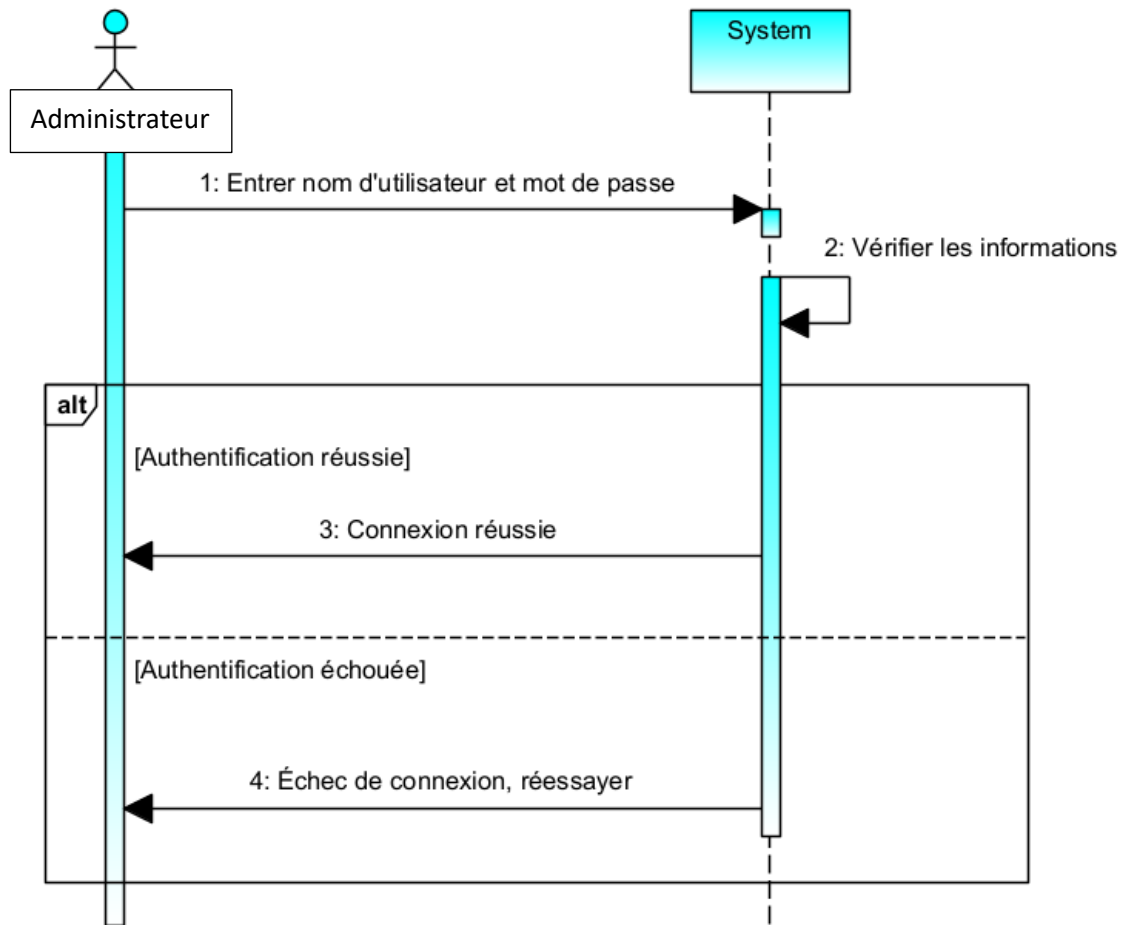


Chapitre 2 : Élaboration – Conception UML

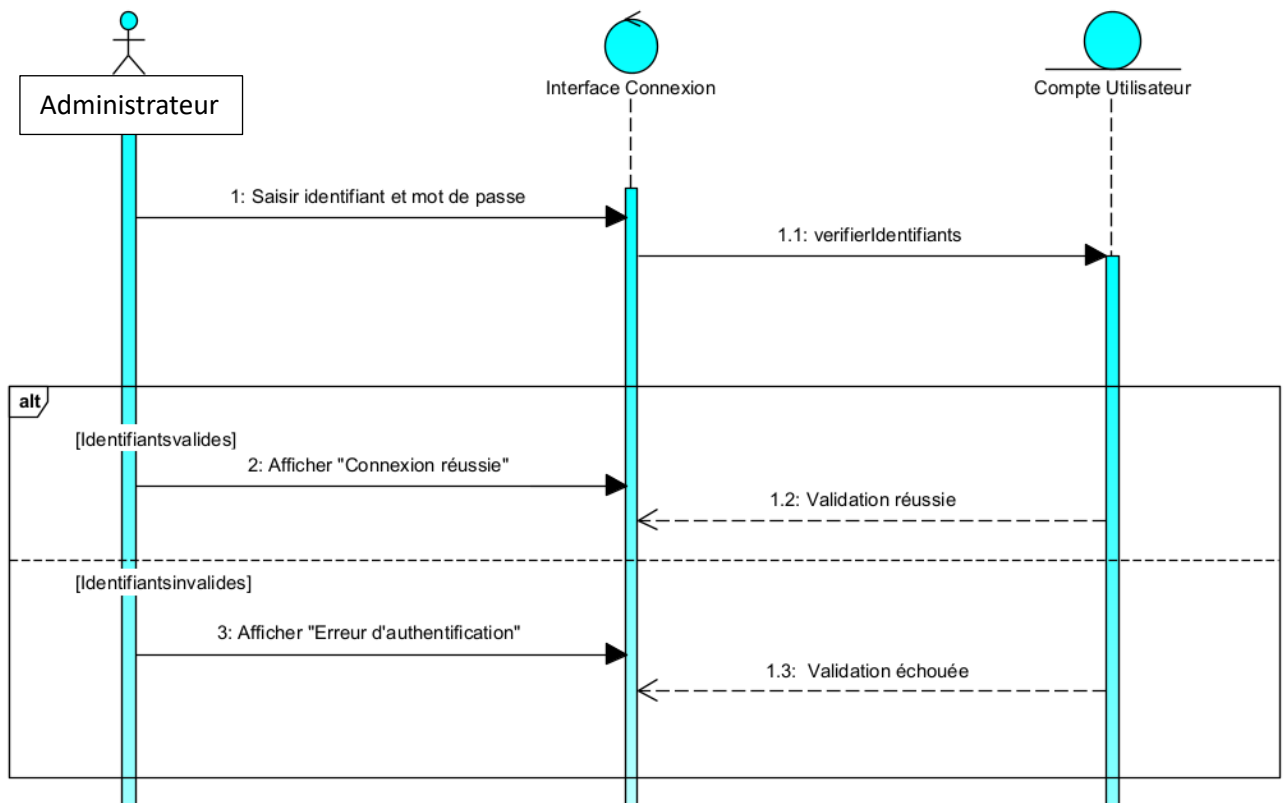
3.1 Diagramme de Cas d'Utilisation (UC1 – S'authentifier)



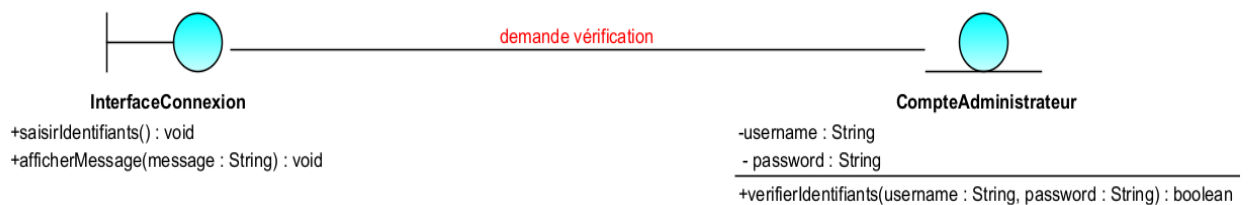
2. Diagramme de Séquence



3. Diagramme de Séquence d'Interaction

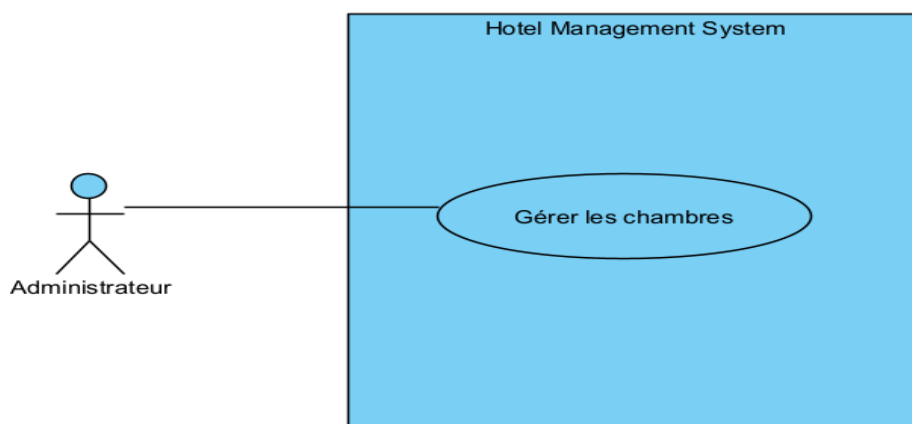


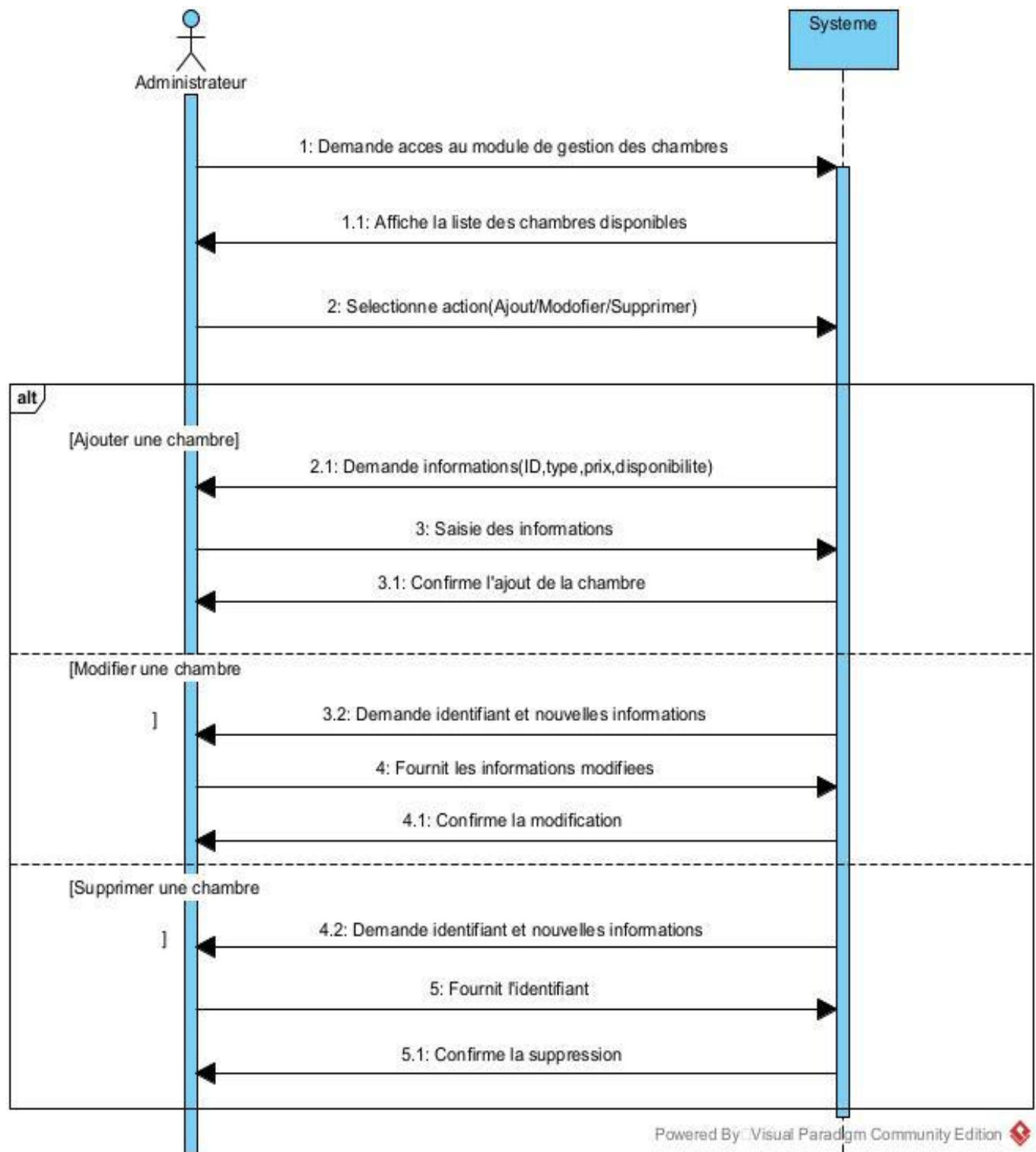
4. Diagramme de Classes Participantes



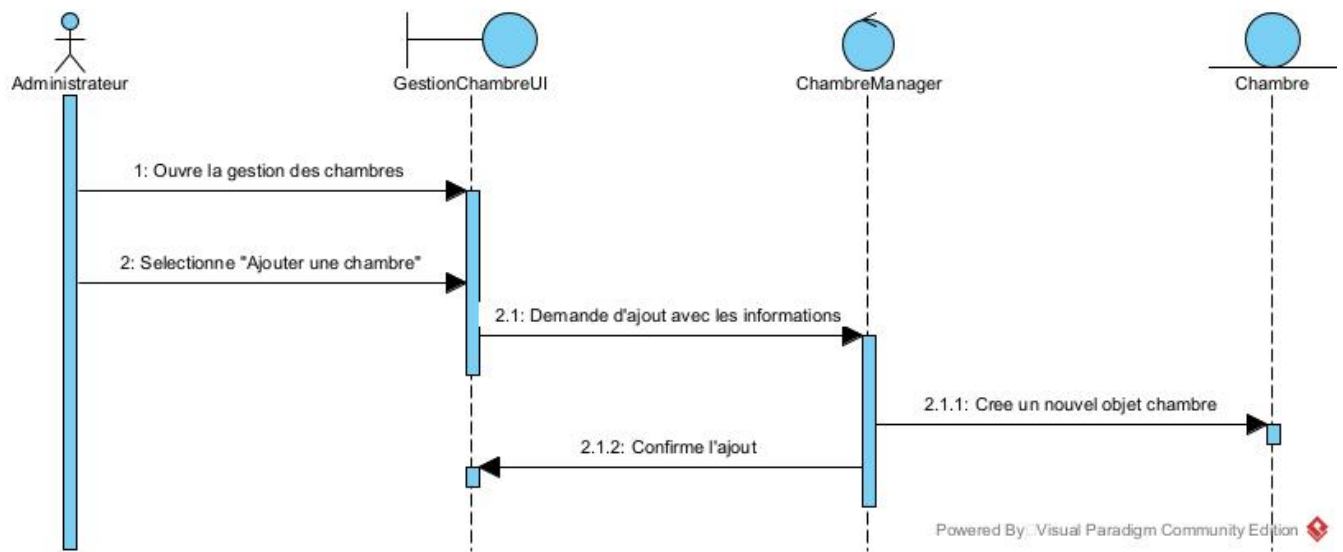
UC2 – Gérer les chambres

1. Diagramme de Cas d'Utilisation

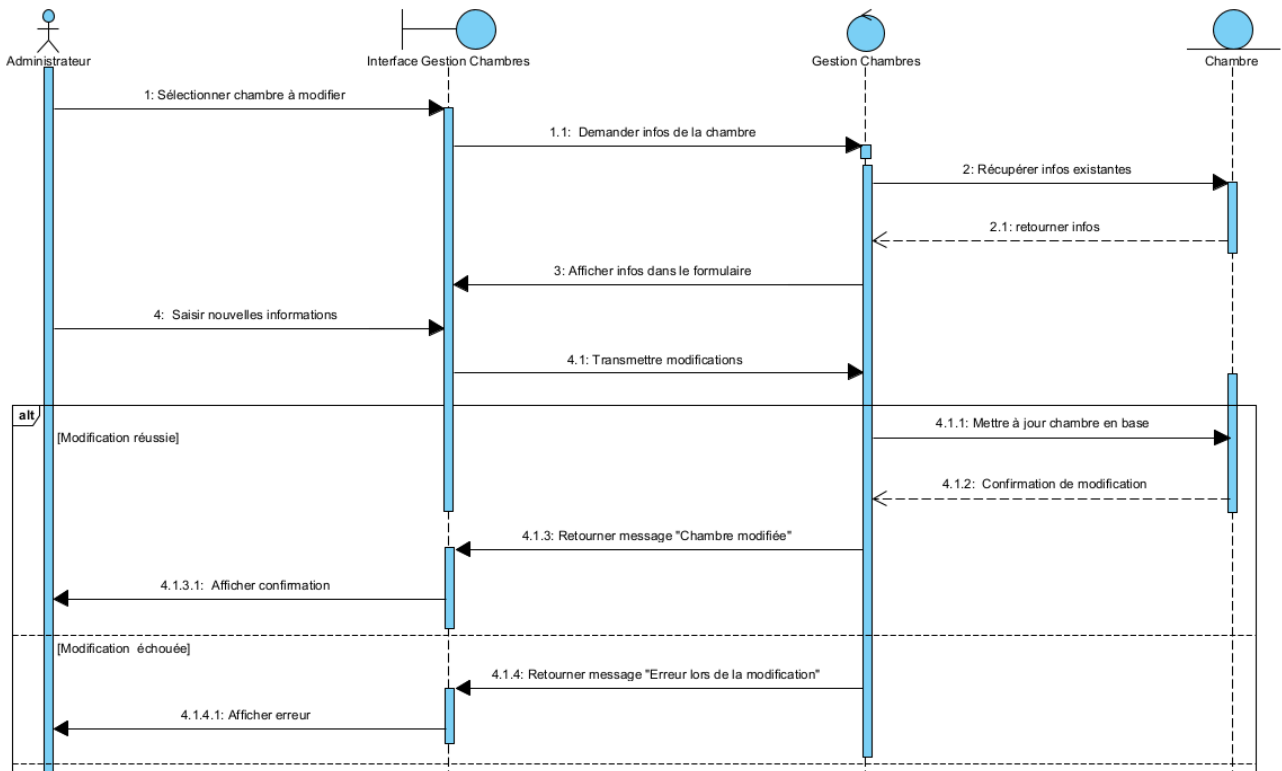




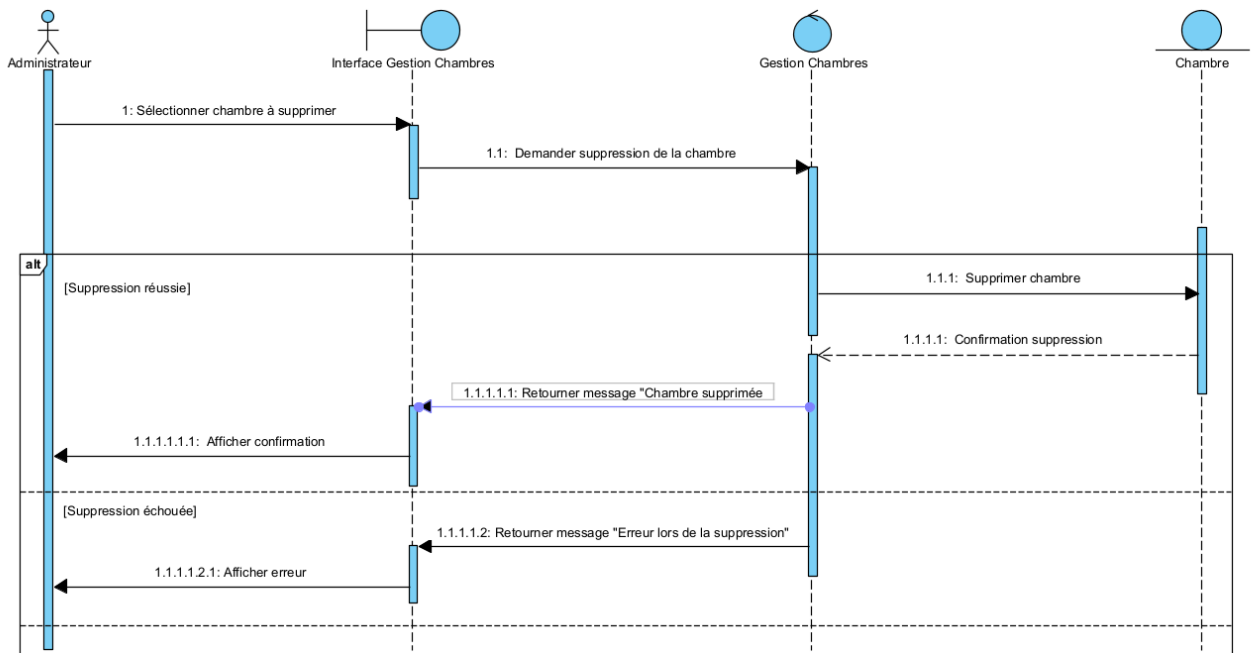
Ajout :



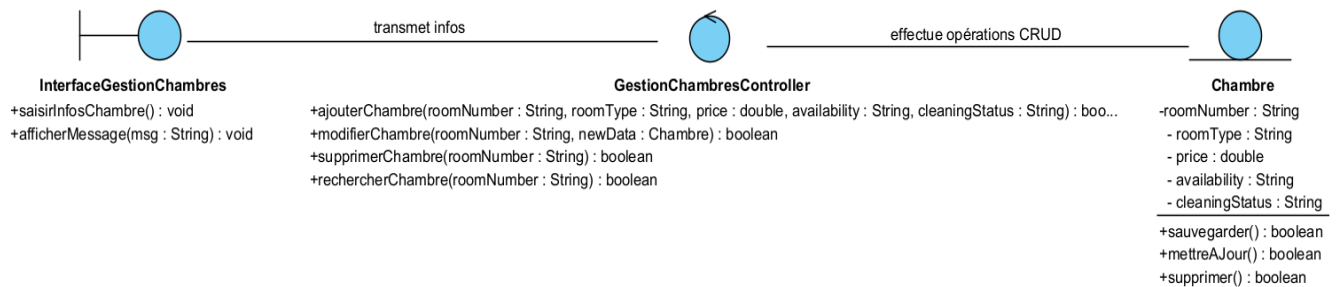
Modification :



Suppression :

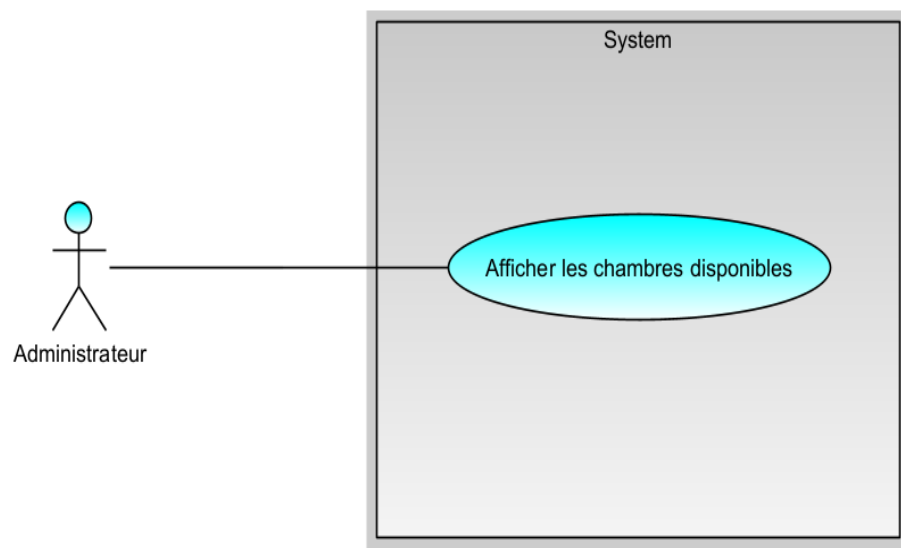


4. Diagramme de Classes Participantes (BCE) :

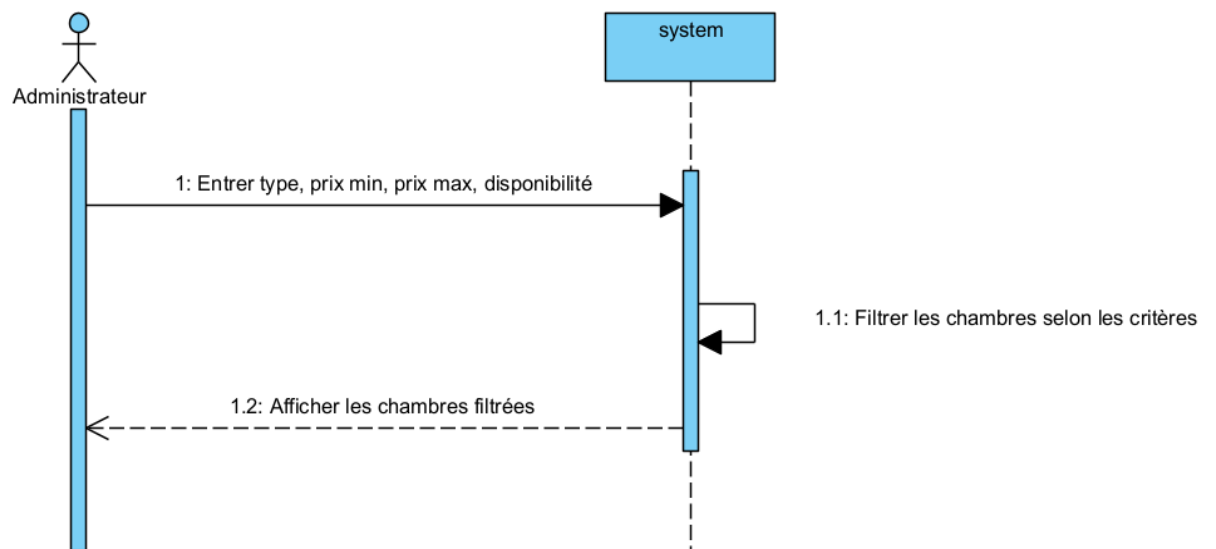


UC3 - Voir les chambres

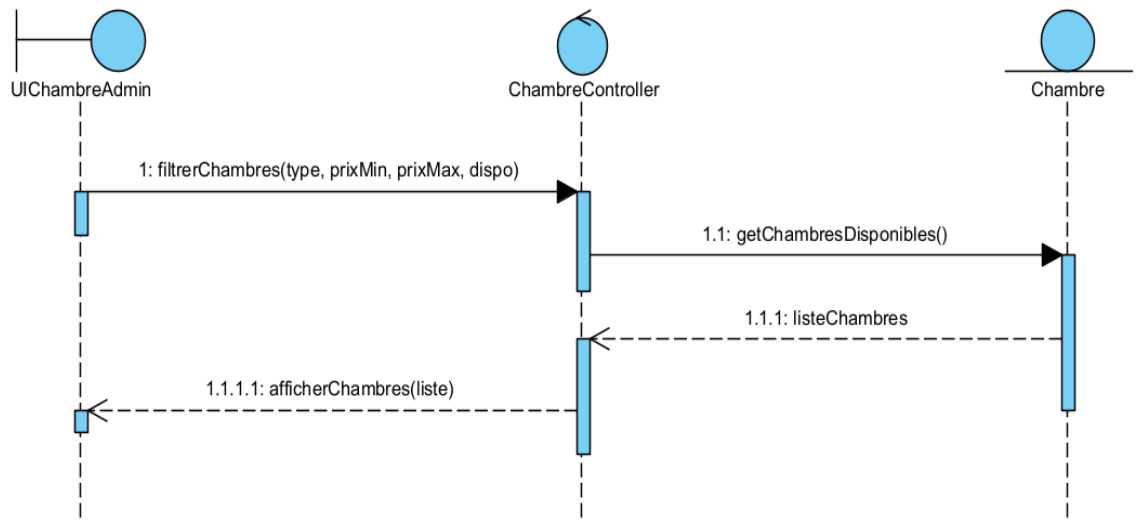
. Diagramme de Cas d'Utilisation :



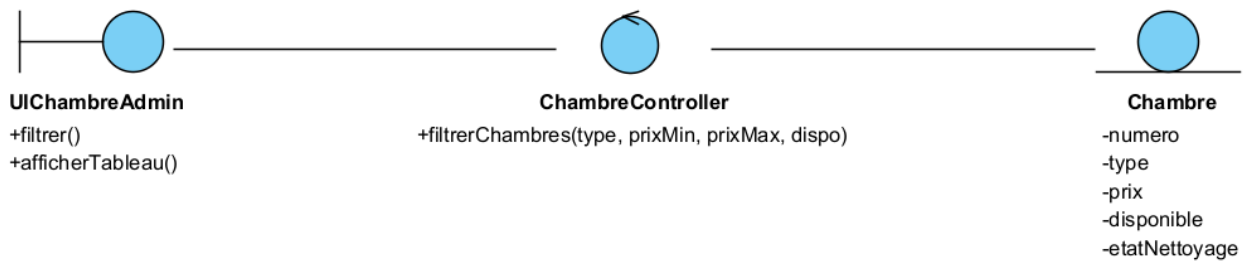
2.Diagramme de Séquence :



3.Diagramme de Séquence d'Interaction :



4. Diagramme de Classes Participantes (BCE) :



Chapitre 4 : Implémentation & Tests

1. Objectifs des Tests

L'objectif des tests est de vérifier la validité du comportement des fonctionnalités principales, notamment :

- La fiabilité de l'authentification.
- La bonne exécution des opérations de gestion des chambres.
- L'affichage correct des chambres disponibles selon les filtres.
- La robustesse du système face aux entrées incorrectes (ex. : mauvais mot de passe).

2. Types de Tests Réalisés

Type de test	Outil utilisé	Description
Test unitaire	JUnit 5	Vérification des méthodes (login, ajout chambre, filtre de disponibilité).
Test de couverture	Jacoco	Mesure de la couverture des classes et méthodes testées.
Analyse de qualité	SonarLint	Détection des mauvaises pratiques (code mort, exceptions mal gérées, etc).

3. Exemples de Scénarios Testés (JUnit)

✓ UC1 – S'authentifier

- ☒ Test avec bons identifiants → Accès autorisé.
- ☒ Test avec mauvais mot de passe → Message d'erreur.
- ☒ Test avec utilisateur inexistant → Message d'erreur.
- ☒ **UC2 – Gérer les chambres**
- ☒ Ajouter une chambre → Chambre présente dans la base.
- ☒ Modifier une chambre → Données mises à jour.
- ☒ Supprimer une chambre → Chambre supprimée avec succès.
- ☒ **UC3 – Voir les chambres**
- ☒ Filtre par type → Affiche uniquement les types demandés.
- ☒ Filtre par prix → Affiche les chambres dans la bonne tranche.
- ☒ Filtre par disponibilité → Affiche les chambres disponibles.

4. Résultats

Tous les tests **JUnit ont été passés avec succès** (BUILD SUCCESS).

Jacoco a permis d'atteindre une **couverture de plus de 80%** sur les classes métier.

SonarLint n'a signalé aucune erreur critique après nettoyage du code.

Interface HOMEFrame

Login



LOGINEFrame

Login

Nom d'utilisateur:

Mot de passe:

Se Connecter

A colorful illustration of a blue hotel building with a sign that says "HOTEL". In front of the building, three people are standing: a man on the left holding a large yellow sign with a thumbs-up icon, a woman in the middle, and a man on the right with a suitcase. The background features stylized pink and purple foliage.

Page AccueilAdmin

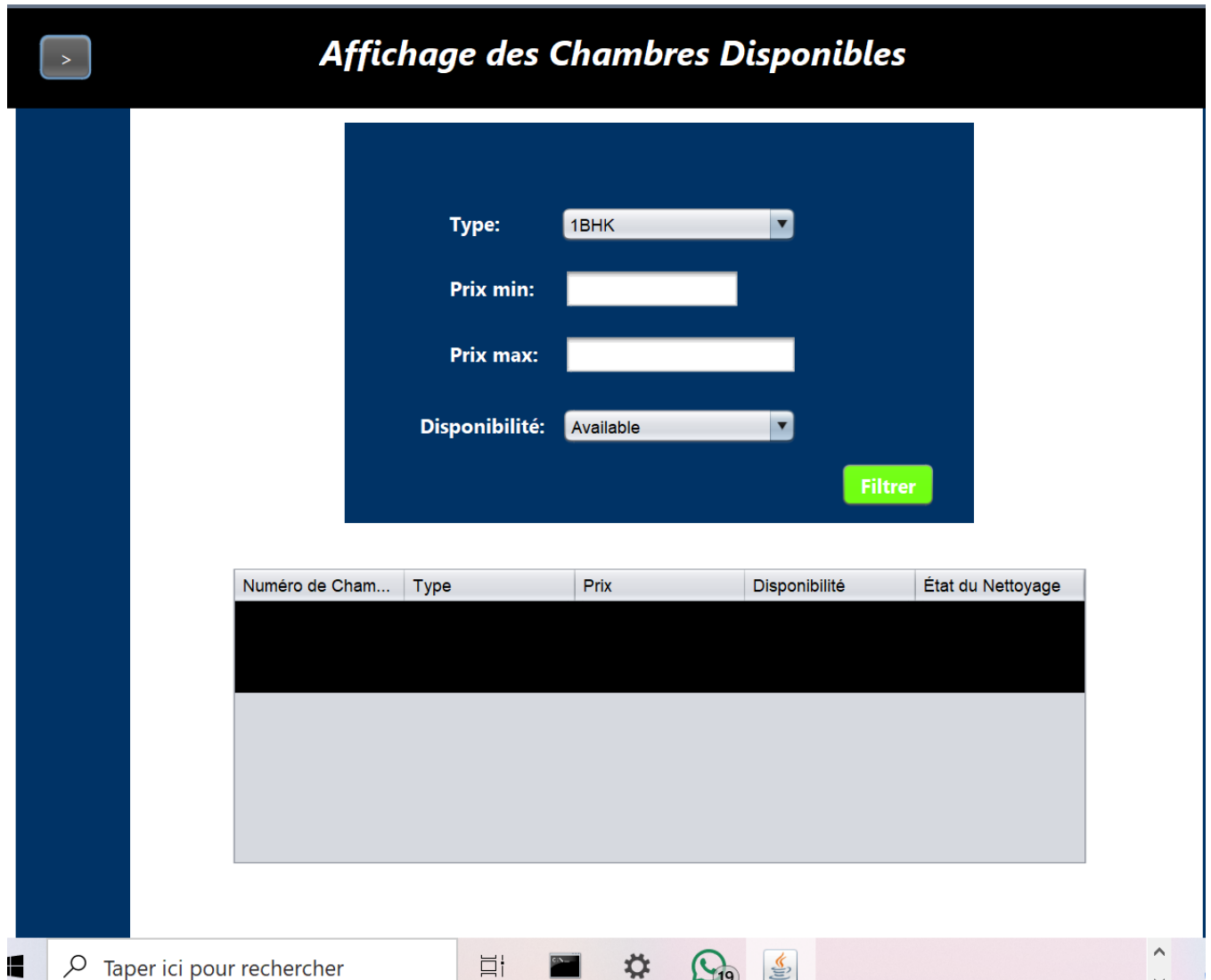
Bienvenue, Admin !

Gérer les chambres

Voir les chambres



Affichagechambres



Gestionchambres

<

Gestion Des Chambres

N° Chambre :

Nettoyage :

Clean

Type :

1BHK

Prix MAD :

Disponibilité :

Available

Ajouter

Modifier

Supprimer

N° Chambre	Type	Prix MAD	Disponibilité	Nettoyage
5	2BHK	3000.0	Available	Clean
8	2BHK	4000.0	Available	Clean
13	1BHK	3000.0	Available	Clean
14	1BHK	400.0	Available	Clean
15	1BHK	300.0	Available	Clean
16	2BHK	6000.0	Occupied	Clean
23	2BHK	34000.0	Available	Clean
123	1BHK	12000.0	Available	Clean
200	1BHK	40.0	Available	Clean
233	2BHK	1000.0	Available	Clean
234	1BHK	200000.0	Available	Clean
400	2BHK	20000.0	Occupied	Clean
600	2BHK	20000.0	Available	Clean
670	2BHK	20000.0	Available	Clean

Conclusion

Ce mini projet m'a permis de mettre en œuvre un système complet de gestion hôtelière en appliquant la méthodologie RUP et en respectant une architecture BCE (Boundary-Control-Entity).

Les tests unitaires, les analyses de couverture avec Jacoco et l'analyse de qualité du code avec Sonar Lint ont contribué à assurer la robustesse et la fiabilité du système.

Enfin, bien que le passage à Maven ait représenté un défi, le projet global répond aux exigences fonctionnelles et non fonctionnelles définies au départ.